Priority queues in Dijkstra's algorithm

41052 - Advanced Algorithms – Programming Assignment 2

Oscar Chevalier Student ID: 24834347

Truong Giang Nguyen Student ID: 25520038

Tung Nguyen Student ID: 24666048

Contents

1 Problem Statement & Algorithm Choices **3**
- 1.1 The problem . . . 3
- 1.2 Dijkstra's algorithm and the role of the heap . . . 3
- 1.3 Algorithm choices . . . 4
  - 1.3.1 Binary heap . . . 4
  - 1.3.2 Pairing heap . . . 5
  - 1.3.3 Fibonacci heap . . . 5
  - 1.3.4 Brodal-Okasaki heap . . . 5

2 Theoretical Comparison and Predictions **6**
- 2.1 Complexity analysis . . . 6
  - 2.1.1 Heap operations . . . 6
  - 2.1.2 Node memory . . . 6
  - 2.1.3 Lazy vs. Decrease-Key . . . 7
- 2.2 Experiment hypotheses . . . 8
  - 2.2.1 Hypotheses for Experiment 1: Running time . . . 8
  - 2.2.2 Hypotheses for Experiment 2: Cache Behaviour . . . 8
  - 2.2.3 Hypotheses for Experiment 3: Memory Management . . . 9

3 Experiment Methodology **9**
- 3.1 Overview . . . 9
- 3.2 Input Graphs . . . 10

4 Results and Analysis **11**
- 4.1 Overview . . . 11
- 4.2 Runtime Results . . . 11
  - 4.2.1 Runtime Comparison Between Heap Implementations . . . 11
  - 4.2.2 Runtime Comparison Between Lazy and Decrease-Key Dijkstra . . . 14
- 4.3 Hardware Performance Results . . . 15
  - 4.3.1 Work and Execution Efficiency . . . 15
  - 4.3.2 Cache and Branch Behaviour . . . 16
- 4.4 Memory Results . . . 17
  - 4.4.1 Peak Heap and Resident Memory . . . 17
  - 4.4.2 Allocator Activity . . . 18
- 4.5 Hypotheses Revisited . . . 19
- 4.6 Cross-Experiment Analysis . . . 19

5 What Surprised Us **20**
- 5.1 Decrease-key is rarely called . . . 20
- 5.2 Brodal-Okasaki's decrease-key is lazy Dijkstra with extra work . . . 20
- 5.3 The heap with the fewest cache misses had the highest miss rate . . . 21
- 5.4 Our measurements were wrong twice before they were right . . . 21
- 5.5 What held . . . 21

6 Future Improvement **21**

7 Individual Contributions and Reflections **22**
7.1 Oscar Chevalier . . . 22
7.2 Truong Giang Nguyen . . . 22
7.3 Tung Nguyen . . . 23
7.3.1 Pairing Heap . . . 23
7.3.2 Fibonacci Heap . . . 24

8 AI Disclosure **24**
8.1 Oscar's Experience . . . 25
8.2 Truong Giang's Experience . . . 25
8.3 Tung's Experience . . . 26
8.4 AI in Group Work . . . 27

1 Problem Statement & Algorithm Choices

1.1 The problem

The object of study is the single-source shortest path (SSSP) problem on directed graphs with non-negative edge weights. The input is a directed graph $G = (V, E)$ with $n = |V|$ vertices and $m = |E|$ directed edges, a weight function $w : E \to \mathbb{Z}_{\geq 0}$, and a source vertex $s \in V$. The weight of a directed path $p = \langle v_0, v_1, \ldots, v_k \rangle$ is the total weight of all edges along the path: $w(p) = \sum_{i=1}^{k} w(v_{i-1}, v_i)$. The shortest path distance from s to v is defined as:

$$dist(s, v) = \begin{cases} \min\{\, w(p) : p \text{ is a path from } s \text{ to } v \,\} & \text{if such a } p \text{ exists,} \\ \infty & \text{otherwise.} \end{cases}$$

The required output is the full distance vector: an array $\texttt{dist}[0 \ldots n-1]$ where $\texttt{dist}[v] = dist(s, v)$ for every reachable v, and a sentinel value of -1 when no path from s to v exists ($dist(s, v) = \infty$). Path traces are not reconstructed, since that process is identical across every implementation and would add a constant overhead to all measurements without discriminating between the priority queues.

The correctness of Dijkstra's algorithm relies on the greedy invariant that once a vertex with the minimum tentative distance is extracted from the priority queue, its shortest path distance is finalised. This property holds strictly when all edge weights are non-negative ($w(e) \geq 0$). In our benchmark suite, all generated edge weights are drawn as strictly positive integers ($w : E \to \mathbb{Z}^+$) with a fixed pseudo-random seed, ensuring no 0-weight cycles exist and guaranteeing that all implementations are evaluated on identical inputs.

1.2 Dijkstra's algorithm and the role of the heap

Author: Truong Giang Nguyen

Dijkstra's algorithm [3] maintains a distance vector $\texttt{dist}[v]$ for every vertex $v \in V$. It initialises $\texttt{dist}[s] = 0$ for the source vertex s and $\texttt{dist}[v] = \infty$ for all other vertices. At each step, it extracts the unsettled vertex u with minimum tentative distance, treats $\texttt{dist}[u]$ as finalised, and relaxes each outgoing directed edge (u, v) if $\texttt{dist}[u] + w(u, v) < \texttt{dist}[v]$. The priority queue is a min-heap, which is the data structure under test in our benchmarks.

We implemented two variants of Dijkstra's algorithm that differ only in how they update the queue after a successful relaxation:

1. **Lazy formulation:** Inserts a new (d', v) vertex–distance pair every time an edge (u, v) is relaxed, leaving any older entries for the same vertex in the heap. When a stale entry for vertex u is later extracted, its extracted distance d is strictly greater than $\texttt{dist}[u]$, so it is skipped. The heap may therefore hold more than one entry per vertex.

2. **Eager formulation (`decrease_key`):** Keeps at most one entry per vertex in the queue. A vertex is pushed when it is first reached ($\texttt{dist}[v] = \infty$), and later improvements call the heap's `decrease_key()` operation to lower the key of the existing entry. We include this variant because some heaps, such as Fibonacci heaps, are designed to make this operation cheap.

Algorithm 1 Dijkstra's algorithm, lazy formulation

```
1: dist[v] ← ∞ for all v ∈ V,    dist[s] ← 0
2: Q ← Heap-Create(),    Push(Q, (0, s))
3: while Q is not empty do
4:    (d, u) ← Min(Q),    Pop(Q)
5:    if d > dist[u] then
6:       continue {stale entry}
7:    end if
8:    for each directed edge (u, v) with weight w(u, v) do
9:       d′ ← d + w(u, v)
10:      if d′ < dist[v] then
11:         dist[v] ← d′
12:         Push(Q, (d′, v))
13:      end if
14:   end for
15: end while
```

The algorithm performs a fixed amount of work outside the queue. Each reachable vertex is settled once and each of its outgoing edges is examined once, which accounts for $\Theta(n + m)$. Everything else is queue work. A vertex's outgoing edges are scanned only the first time it is extracted, because stale entries are skipped before the loop. Every directed edge is therefore relaxed at most once, so there are at most m relaxation pushes plus the initial source push. Every pushed entry is eventually extracted, giving at most $m + 1$ pops, of which at most n settle a vertex and the rest are stale entries. Eventually, the running time is:

$$\mathcal{O}(n + m + (m + 1)\, T_{\text{PUSH}} + (m + 1)\, T_{\text{POP}})$$

Algorithm 2 Dijkstra's algorithm, eager update (`decrease_key`)

```
1: dist[v] ← ∞ for all v ∈ V,    dist[s] ← 0
2: Q ← Heap-Create(n),    Push(Q, (0, s))
3: while Q is not empty do
4:    (d, u) ← Min(Q),    Pop(Q)
5:    for each directed edge (u, v) with weight w(u, v) do
6:       d′ ← d + w(u, v)
7:       if d′ < dist[v] then
8:          if dist[v] = ∞ then
9:             Push(Q, (d′, v))
10:         else
11:            Decrease-Key(Q, (d′, v))
12:         end if
13:         dist[v] ← d′
14:      end if
15:   end for
16: end while
```

For the eager variant, the work outside the queue is the same $\Theta(n+m)$, but the queue work changes. The queue only pushes vertices that are not yet encountered, so there are at most n pushes and n pops. `decrease_key()` is invoked when an edge relaxation successfully discovers a shorter path to an already-discovered vertex, so it is triggered at most $m - n + 1$ times. Hence the running time:

$$\mathcal{O}(n + m + n \cdot T_{\text{PUSH}} + n \cdot T_{\text{POP}} + (m - n + 1) \cdot T_{\text{DEC}})$$

1.3 Algorithm choices

1.3.1 Binary heap

Author: Truong Giang Nguyen

The binary heap [7] represents a complete binary tree of n items stored implicitly in a flat array, with the children of index i at $2i+1$ and $2i+2$ and the parent at $\lfloor (i-1)/2 \rfloor$. The heap invariant is that no node's

key is smaller than its parent's, so the minimum is always at index 0 and is read in $\mathcal{O}(1)$ time. Insertion appends at the end of the array and *bubbles* the item towards the root while it is smaller than its parent. Extraction overwrites the root with the last item and *sinks* it downwards, swapping with its smaller child while the invariant is violated. Both paths are bounded by the tree height, giving $\mathcal{O}(\log n)$ worst-case insertion, extraction, and key decrease. The tree structure is based on index arithmetic rather than pointers, so the entire heap occupies one contiguous allocation. A separate index array `pos`$[v]$ records where vertex v currently sits so that DECREASE-KEY can locate it in $\mathcal{O}(1)$ time before bubbling.

1.3.2 Pairing heap

Author: Tung Nguyen

A pairing heap [4] is a self-adjusting multi-way tree implemented using a Left-Child Right-Sibling (LCRS) layout. Each node stores three pointers: `child`, `next`, and `prev` (which references its left sibling, or its parent if the node is the leftmost child, enabling $\mathcal{O}(1)$ detachment). Tree combinations rely on MELD, which links the root carrying the larger key as the leftmost child of the smaller root in $\mathcal{O}(1)$ time. Consequently, PUSH melds a freshly allocated node into the root, while DECREASE-KEY cuts the target node from its siblings via `pos`$[v]$ and melds it into the root in $\mathcal{O}(1)$ immediate time (with an amortised cost of $o(\log n)^*$) without invalidating other vertex handles. All structural rebalancing is deferred lazily to POP: after severing the root, its orphaned children are coalesced via a *two-pass pairing* scheme (pairing adjacent siblings left-to-right, then accumulating them right-to-left) to achieve $\mathcal{O}(\log n)$ amortised extraction. Compared to the array-backed binary heap, the pairing heap trades cache locality and per-node dynamic allocation overhead for cheap, pointer-based decrease-key operations.

1.3.3 Fibonacci heap

Author: Tung Nguyen

A Fibonacci heap [5] is a collection of min-heap-ordered trees maintained in a circular doubly-linked root list, with a direct pointer to the minimum root. Each node stores four pointers (`parent`, `child`, `left`, `right`), an integer `degree`, and a boolean `marked` flag indicating whether the node has lost a child since becoming a child of its current parent. Operations are structured around lazy consolidation. PUSH splices a new node into the root list in $\mathcal{O}(1)$ worst-case time. When DECREASE-KEY violates heap order, the node is severed and appended to the root list via CUT. If its parent was already marked, a *cascading cut* propagates recursively up the ancestor chain until an unmarked node is encountered and marked. This mechanism preserves an exponential lower bound on subtree size relative to degree (F_{k+2} for degree k), achieving an amortised $\mathcal{O}(1)$ decrease-key bound. Structural cleanup occurs exclusively during POP, where the root's children are promoted to the root list and trees of equal degree are pairwise linked using an auxiliary array (`A[MAX_DEGREE]`), yielding $\mathcal{O}(\log n)$ amortised extraction. This achieves the theoretically optimal $\mathcal{O}(m + n \log n)$ bound for Dijkstra's algorithm. However, it incurs heavy per-node structures (56 bytes plus handles), complex cascading cut logic, and poor cache locality.

1.3.4 Brodal-Okasaki heap

Author: Oscar Chevalier

Traditionally, the Brodal-Okasaki heap [2] is a persistent pure functional heap. It is a bootstrapped skew-link binomial heap, which provides the best possible theoretical performance bounds for a functional heap. There is also an imperative variant [1] that is more similar to the Fibonacci heap, but uses skew-links. Brodal-Okasaki achieves PUSH in $\mathcal{O}(1)$ and POP in $\mathcal{O}(\log n)$; DECREASE-KEY is more complex, but fits in $\mathcal{O}(\log n)$.

Several major adjustments have been made to the Brodal-Okasaki implementation in order to try to improve its performance. While the overall pure functional construction of the heap has been kept, bootstrapping has been removed as the meld operation does not get used in Dijkstra's algorithm. Additionally, we decided not to worry about the purity of the memory management. When the heap is modified, the implementation happily mutates pointers and cells, making the implementation *in-place*. In order to improve cache locality, all the cells are allocated in one contiguous buffer, effectively maintaining a custom free-list / memory allocator for the heap. Cells use relative addresses to point to each other, allowing the buffer to be reallocated without invalidating the contents. Finally, a negative queue has been added to support the decrease-key operation.

2 Theoretical Comparison and Predictions

2.1 Complexity analysis

2.1.1 Heap operations

The theoretical merit of priority queues in shortest-path algorithms can be evaluated by how efficiently they execute three core operations:

- PUSH: inserting an unvisited frontier node.
- POP: extracting the unvisited node with minimum tentative distance.
- DECREASE-KEY: relaxing an edge to an already-discovered node.

Table 1: Asymptotic Complexity and Pointer Overhead per Node

Heap Variant	Push	Pop (Extract)	Decrease-Key	Pointers / Node
Binary	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	0 (implicit array)
Pairing	$\mathcal{O}(1)$	$\mathcal{O}(\log n)^*$	$o(\log n)^*$	3 (`child, next, prev`)
Fibonacci	$\mathcal{O}(1)$	$\mathcal{O}(\log n)^*$	$\mathcal{O}(1)^*$	4 (`parent, child, left, right`)
Brodal-Okasaki	$\mathcal{O}(1)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)^\dagger$	0 (buffer indices)

*Aggregate per-operation cost bound. †Simulated via positive/negative tree cancellation.

2.1.2 Node memory

Table 2: Memory Layout and Struct Size per Node (64-bit Architecture)

Heap Variant	Pointers / Indices	Padding	Total Node Size
Binary	0 B	0 B	**16 B**
Pairing	24 B (3 pointers)	0 B	**40 B**
Fibonacci	32 B (4 pointers)	3 B	**56 B**
Brodal-Okasaki	24 B (3 `size_t`)	0 B	**40 B**

Note: The binary, pairing, and Fibonacci heaps additionally maintain an external vertex lookup array (`pos`), adding 8 bytes per vertex (a `size_t` index for Binary, or a `Node*` handle for pointer heaps). Brodal-Okasaki has no such array: its decrease-key identifies an entry by its (distance, vertex) value.

On standard 64-bit systems (`x86_64`), pointers and `size_t` primitives are 8 bytes in size and require 8-byte alignment boundaries. The physical memory footprint of each node is determined as follows:

1. **Payload (`Item`):** The fundamental payload stored by all heaps is the `Item` struct (`int distance`, `size_t vertex`). While `distance` requires only 4 bytes, the subsequent 8-byte `vertex` field forces the compiler to insert 4 bytes of tail padding to maintain 8-byte alignment, bringing `sizeof(Item)` = 16 bytes.
2. **Binary Heap:** The binary heap allocates no per-node wrapper structs or metadata. Items reside contiguously in a flat array (`Item data[]`), consuming exactly 16 bytes per element. When paired with the external index mapping array (`size_t pos[]`, 8 bytes per vertex), the total memory allocated per active vertex is exactly:
$$16 \text{ B} + 8 \text{ B} = 24 \text{ B/vertex}$$
3. **Pairing Heap:** Each `Node` contains the 16-byte `item` payload followed by three 8-byte pointers: `child`, `next`, and `prev`. Because all members are multiples of 8 bytes, no internal structure padding is required:
$$16 \text{ B (item)} + 3 \times 8 \text{ B (pointers)} = 40 \text{ bytes per node}$$
Accounting for the external `Node* pos[]` pointer array yields a combined active footprint of 48 bytes per vertex.

4. **Fibonacci Heap:** A Fibonacci `Node` introduces significant structural metadata:
   - `Item item`: 16 bytes (offsets $0 \dots 15$)
   - `int degree`: 4 bytes (offsets $16 \dots 19$)
   - `bool marked`: 1 byte (offset 20)
   - Compiler alignment padding: 3 bytes (offsets $21 \dots 23$) to align the subsequent pointer fields to an 8-byte boundary.
   - Four pointers (`parent`, `child`, `left`, `right`): $4 \times 8 = 32$ bytes (offsets $24 \dots 55$)

   This sums to `sizeof(Node)` $= 56$ bytes. With the 8-byte external `pos` handle, each settled vertex incurs a memory commitment of 64 bytes.

5. **Brodal-Okasaki Heap:** The cell buffer stores `struct Tree`, consisting of the 16-byte payload, an 8-byte integer `rank`, and two 8-byte relative buffer indices (`child` and `sibling`):

$$16 \text{ B (item)} + 8 \text{ B (rank)} + 2 \times 8 \text{ B (indices)} = 40 \text{ bytes per cell}$$

2.1.3 Lazy vs. Decrease-Key

Table 3: Tradeoffs between Decrease-Key vs. Lazy Dijkstra

Metric / Operation	Decrease-key	Lazy
Max Priority Queue Size	Exactly n vertices	Up to m entries
Total Push Invocations	Exactly n	At most $m+1$
Total Pop Invocations	Exactly n	At most $m+1$
Total Decrease-Key Calls	At most $m-n+1$	0
Handle Tracking (`pos` array)	Required	None
Binary Heap Complexity	$\mathcal{O}(m \log n)$	$\mathcal{O}(m \log m) = \mathcal{O}(m \log n)$
Fibonacci Heap Complexity	$\mathcal{O}(m + n \log n)$	$\mathcal{O}(m \log m) = \mathcal{O}(m \log n)$
Memory Footprint Scaling	$\mathcal{O}(n)$	$\mathcal{O}(m)$

The choice between Decrease-Key and lazy insertion fundamentally alters how priority queue overhead is distributed:

1. **The Position Tracking with Decrease-key:** While eager mode restricts the queue size to at most n items, it imposes a continuous maintenance tax. For array-based binary heaps, every swap during a `bubble()` or `sink()` must perform a secondary write to update `pos[vertex]`. In pointer heaps, Decrease-Key requires unlinking and re-melding subtrees.

2. **The Elimination of Decrease-Key in Lazy Mode:** Lazy completely bypasses the need for node handles and position synchronisation. When an edge (u, v) is relaxed, a fresh `(distance, v)` tuple is appended with a simple Push. This transforms edge relaxation from a complex tree-restructuring operation into a trivial insertion.

3. **The Penalty of Queue Bloat on Dense Graphs:** The vulnerability of the lazy formulation lies in queue inflation on dense topologies ($m \gg n$). Because stale entries linger in the heap until they float to the root, the queue swells from size n to $\mathcal{O}(m)$. Consequently:
   - Every extraction incurs tree depth penalties proportional to $\log m$ instead of $\log n$.
   - The algorithm performs m total extractions rather than n. While skipping a stale entry requires only an $\mathcal{O}(1)$ scalar distance comparison (`d > dist[u]`), getting that stale entry to the top of the heap requires a full, expensive $\mathcal{O}(\log m)$ Pop restructuring.
   - For Fibonacci and pairing heaps, lazy insertion nullifies their asymptotic advantage: their $\mathcal{O}(1)$-like decrease-key is never exercised, reducing their theoretical bound from $\mathcal{O}(m + n \log n)$ back to $\mathcal{O}(m \log m)$.

2.2 Experiment hypotheses

2.2.1 Hypotheses for Experiment 1: Running time

- **Hypothesis 1.1:**
  On sparse graphs with low average degree ($\bar{d} = m/n \leq 4$), edge relaxations are infrequent, meaning Dijkstra's execution is dominated by n vertex extractions (POP). Because binary heap elements are densely packed in contiguous arrays with zero pointer indirection, the binary heap will achieve the lowest wall-clock runtime, outperforming both pairing and Fibonacci heaps despite having an inferior theoretical decrease-key bound.

- **Hypothesis 1.2:**
  As graph density increases ($\bar{d} \gg 32$, $m \gg n$), the number of DECREASE-KEY operations ($m - n$) will overwhelm the n extractions.

  - We predict that the pairing heap will cross over and beat the binary heap on dense topologies due to its lightweight $\mathcal{O}(1)$-like cut-and-meld decrease-key.
  - Conversely, we predict the Fibonacci heap will fail to beat the pairing heap across all graph densities. This is due to the structural tax of maintaining circular doubly-linked lists, degree arrays, and cascading cut bookkeeping.

- **Hypothesis 1.3:**
  The relative performance of LAZY against DECREASE-KEY will depend strictly on edge density:

  - On *sparse graphs*, LAZY will be faster than or strictly competitive with DECREASE-KEY for binary heaps, because avoiding the synchronisation writes to `pos[]` during `bubble()` outweighs the cost of processing a small number of duplicate entries.
  - On *dense graphs*, LAZY will suffer severe performance degradation across all heaps. The queue will swell to size $\mathcal{O}(m)$, forcing the algorithm to execute m full $\mathcal{O}(\log m)$ extractions instead of n, quickly falling behind DECREASE-KEY.

- **Hypothesis 1.4:**
  While purely functional priority queues offer optimal theoretical worst-case bounds, simulating DECREASE-KEY via positive/negative tree cancellation will introduce quadratic-like slowdowns on dense graphs with repeated key updates, rendering it uncompetitive with imperative heaps.

2.2.2 Hypotheses for Experiment 2: Cache Behaviour

- **Hypothesis 2.1:**
  We predict a pronounced gradient in cache miss rates that correlates directly with node size and layout:

  - The **Binary Heap** will exhibit the lowest cache miss rates because traversing an array index ($2i + 1, 2i + 2$) exploits CPU spatial locality.
  - **Pairing and Brodal-Okasaki** will exhibit significantly higher L_1 and Last-Level Cache (LLC) miss rates due to pointer chasing. Since individual nodes are allocated arbitrarily in virtual memory, traversing child and sibling pointers will incur frequent pipeline stalls waiting for DRAM round-trips.
  - The **Fibonacci Heaps** will record the worst cache behaviour because its 56-byte node size approaches an entire 64-byte cache line, minimising the probability that a pointer dereference hits already-cached data.

- **Hypothesis 2.2:**
  Array-based heaps will demonstrate lower Instructions per Cycle (IPC) during bubble/sink loops due to data dependency stalls, but will execute orders of magnitude fewer total retired instructions than pointer-based heaps that must maintain multi-way pointer updates per link.

2.2.3 Hypotheses for Experiment 3: Memory Management

- **Hypothesis 3.1:**
  Peak heap memory usage during eager Dijkstra will strictly mirror the analytical struct calculations:

  $$\text{Peak Memory: Binary} < \text{Brodal-Okasaki} < \text{Pairing} < \text{Fibonacci}$$

  In lazy mode, peak heap memory will scale with edge volume (m) rather than vertex count (n), consuming multiples of the eager memory footprint on dense graphs due to duplicate frontier nodes. Furthermore, pointer heaps will show disproportionately higher Peak Resident Set Size (RSS) due to heap arena fragmentation across scattered 40–56 byte allocations.

- **Hypothesis 3.2:**
  - **Buffer-backed Heaps (Binary & Brodal-Okasaki):** Will incur $\mathcal{O}(1)$ dynamic memory allocations, relying on pre-allocated contiguous buffers and internal free-list management rather than issuing requests to the system allocator.
  - **Pointer Heaps (Pairing & Fibonacci):** Will generate massive allocator churn ($\sim n$ invocations of `malloc()` and `free()` in eager mode, and $\sim m$ in lazy mode), introducing memory manager bookkeeping overhead and cache eviction during node construction.

3 Experiment Methodology

3.1 Overview

All tests were conducted on a machine equipped with an Intel Core Ultra 7 165U processor, 12 physical cores and 14 logical CPUs, 30 GiB of RAM, and a 937 GB NVMe solid-state drive. The machine was running 64-bit *Arch* Linux with kernel version 7.2.11-arch1-1. All code was compiled with GCC using `-O3 -std=c11`. The final measurements were taken on mains power with the operating system's performance power profile; an earlier run on default settings let the processor throttle partway through one benchmark (Section 5). The complete test suite was executed on Linux, while macOS supports only the runtime benchmarks because the `perf` and memory benchmarks depend on Linux-specific facilities. Due to high variance out of our control we do not expect every machine to produce the same results, or even subsequent runs to produce the same results, but they should all be relatively similar.

We tested each heap implementation against two Dijkstra implementations and several input graphs, gathering data about the running time, cpu behaviour, and memory use. In order to reuse code for each task we used `make` to generate several executables, with each executable testing one aspect of a particular heap implementation, each executable links common objects (`dijkstra.o`, `graph.o`), the input graph objects (`graph_registry.o`, `uniform_sparse.o`, `uniform_medium.o`, ...), a specific heap implementation's object(s) (`binary.o`, `fibonacci.o`, ...), and an entry point (`runtime/main.o`, `perf/main.o`, `mem/main.o`). Thus producing an executable that will test the specified performance aspect of a particular heap for all the input graphs.

We used python to generate input graphs as source C files, enabling the produced executables to automatically discover the input graphs. This allows most of the test options to be derived from the Makefile. Random number generation during this process is done using a fixed seed, ensuring that the input data is always the same across different machines and builds. Compiling and linking the large graphs means that fresh build times are higher, but all the benchmarks run quickly, as there isn't additional overhead of reading and parsing large input files.

Reproducibility was a focus of the test setup. A guix manifest file allows a user to get all the dependencies of the project on Linux and run the full pipeline in an isolated container easily.

We used `ubench`, a single header C benchmarking library to run our runtime benchmarks. `ubench` first times a single run to choose a batch of between 10 and 500 runs (about 100 ms of work), then repeats batches of increasing size until the 99% confidence interval of the mean is within 5%. This allows us to gather a good estimate of the running time of each heap implementation in both `decrease_key` and `lazy` Dijkstra implementations. We took care to ensure that the benchmarks were only measuring the running time of Dijkstra over the heap implementation, and that we were not including static overheads due to test initialisation or additional allocations into the benchmarks. Before timing, each runtime benchmark runs Dijkstra once and compares a checksum of the resulting distance array with one computed by an independent Python implementation when the graph was generated, so every timed configuration is also verified to be correct. This pass also warms the caches before the timed runs begin.

The runtime and `perf` benchmarks are pinned to a single CPU with `taskset`. On a hybrid processor such as the Core Ultra 7 165U the type of core decides the caches a run sees (a P-core has a 48 KiB L1 data cache and its own 2 MiB L2, while the E-cores have smaller L1 caches and share an L2), so `src/bench/bench_cpu.py` selects a performance core automatically. On machines without `taskset`, such as macOS, the benchmarks run unpinned.

On Linux, we use the `perf` tool to measure data about the cycles, branching, and cache loads and misses that occur as Dijkstra runs. `perf` starts with its counters disabled, and the benchmark binary enables them only around the timed Dijkstra loop, so process start-up, graph construction and a warm-up run are excluded. Each configuration is measured over 50 runs and the counts are reported per run. We were interested in measuring the reasons behind the performance differences, and in modern processors cache locality has a massive impact on program performance. This is where the difference between the theoretical big-$\mathcal{O}$ performance of each heap and the reality will be made apparent. We use a python script to run the binaries and collate the produced data.

Also on Linux we do a separate test for memory usage, this time wrapping the stdlib's `malloc`, `calloc`, `realloc`, and `free` with custom functions to capture the allocation behaviours and memory usage of each heap. This data is related to the cache performance measured in `perf`, but measured using totally different techniques, so we are looking to compare the data from `mem` against the data from `perf` to see how allocations relate to cache locality. This benchmark also uses a python script to run the binaries and collate the produced data.

Each test (`runtime`, `perf`, `mem`) produces csvs of the results, which we then process into svg graphics and TEX files using more python scripts, allowing them to be easily embedded into the report.

3.2 Input Graphs

To compare the priority queues under different workload shapes, we use seven deterministically generated directed graphs. The graph definitions are kept in the Makefile, while `src/bench/gen.py` generates the corresponding C source files. Every graph uses the fixed seed `67`, positive integer edge weights drawn uniformly from 1 to 100 (1 to 50 for `smoke_1k`), and the same generated input for every heap implementation and Dijkstra variant. Dijkstra always starts from vertex 0.

The suite combines uniform random graphs with structured graphs. The uniform graphs vary the number of vertices and average out-degree, allowing us to observe how the priority queues behave as the number of edge relaxations increases. The structured graphs represent workload patterns that are not captured well by uniform random sampling, such as bounded-degree spatial networks, concentrated hubs, and broad layered frontiers.

Table 4: Generated input graphs used in the benchmarks.

Name	Family	Size
`smoke_1k`	Uniform	$n = 1{,}000,\ m = 4{,}000$
`uniform_sparse`	Uniform	$n = 10{,}000,\ m = 40{,}000$
`uniform_medium`	Uniform	$n = 20{,}000,\ m = 200{,}000$
`uniform_dense`	Uniform	$n = 10{,}000,\ m = 500{,}000$
`grid_224`	Grid	$224 \times 224,\ n = 50{,}176$
`hub_skewed`	Hub-skewed	$n = 20{,}000,\ m = 120{,}000$
`layered_frontier`	Layered	$100 \times 200,\ n = 20{,}000$

Table 5: Purpose of each generated input graph.

Name	Purpose
`smoke_1k`	Small build and correctness check
`uniform_sparse`	Sparse workload; average out-degree 4
`uniform_medium`	Intermediate relaxation workload; average out-degree 10
`uniform_dense`	Dense random workload; average out-degree 50
`grid_224`	Bounded-degree spatial graph
`hub_skewed`	Concentrated traffic around 32 hub vertices
`layered_frontier`	Broad frontier with forward fan-out 8

The smoke_1k graph is included as a small and inexpensive validation case. It allows graph construction, executable linking, checksum validation, and benchmark execution to be checked without the cost of the larger inputs. The three uniform graphs provide a controlled progression from sparse to dense connectivity. Their increasing average out-degree produces more edge relaxations and changes the amount of work performed by the priority queue.

The uniform graphs sample m distinct ordered vertex pairs without replacement, so they contain no self-loops or parallel edges. The grid_224 graph is a directed four-neighbour grid. Each interior vertex has up to four outgoing edges, producing a bounded-degree graph with spatial structure and longer paths than the uniform random graphs. The hub_skewed graph contains a concentrated set of 32 hubs together with background edges: every other vertex has an edge to and from hub $u \bmod 32$, and each remaining edge has a random source and, with probability 0.75, a hub as its target. This creates highly uneven vertex degrees and bursts of relaxation activity. Finally, layered_frontier contains 100 layers of 200 vertices, with deterministic forward edges between consecutive layers: each vertex has edges to 8 vertices of the next layer chosen by a fixed pattern, so only the weights are random. Its broad active frontier is useful for observing heap growth and the difference between eager decrease-key and lazy duplicate-entry Dijkstra.

All seven graphs are used in every benchmark category. The generated graph sources are written below build/test/, and the generated registry and metadata are written to build/test/graph_registry.c and build/test/graphs.json. The graph objects are linked directly into the runtime, perf, and memory benchmark executables, so the benchmark runs do not include graph-file parsing or input-loading overhead.

4 Results and Analysis

4.1 Overview

This section presents the experimental results for the four priority queue implementations used by Dijkstra's algorithm: binary, pairing, Fibonacci, and Brodal-Okasaki heaps. Each implementation was evaluated using the seven input graphs described in Section 3.2 and both Dijkstra variants: eager decrease_key and lazy duplicate-entry processing.

We first compare execution time, both between heap implementations and between the two Dijkstra variants. We then examine hardware performance counters and memory measurements to identify factors that may explain the observed runtime differences. The results are discussed with respect to graph size, density, and structure, including uniform random, grid, hub-skewed, and layered graphs.

4.2 Runtime Results

4.2.1 Runtime Comparison Between Heap Implementations

Table 6 reports the mean execution time of each heap implementation across the seven input graphs. The corresponding graph-specific runtime plots are shown in Figure 1.

Table 6: Dijkstra Execution Time across Heaps and Input Datasets (Mean ± 99% Confidence)

Dataset	**Mode**	**binary**	**brodal-okasaki**	**fibonacci**	**pairing**
smoke_1k	Decrease-Key	30.99 µs	228.21 µs	137.81 µs	46.95 µs
	Lazy	36.97 µs	184.16 µs	176.34 µs	58.05 µs
uniform_sparse	Decrease-Key	718.96 µs	3.46 ms	2.01 ms	925.24 µs
	Lazy	873.17 µs	2.82 ms	2.36 ms	1.08 ms
uniform_medium	Decrease-Key	2.13 ms	16.24 ms	4.86 ms	3.10 ms
	Lazy	3.25 ms	10.49 ms	8.04 ms	4.84 ms
uniform_dense	Decrease-Key	1.69 ms	16.11 ms	3.08 ms	2.43 ms
	Lazy	3.18 ms	9.57 ms	7.04 ms	4.26 ms
grid_224	Decrease-Key	2.40 ms	12.49 ms	7.23 ms	3.58 ms
	Lazy	2.75 ms	10.13 ms	8.99 ms	4.14 ms
hub_skewed	Decrease-Key	1.54 ms	6.43 ms	4.09 ms	2.53 ms
	Lazy	1.72 ms	5.59 ms	4.52 ms	2.80 ms
layered_frontier	Decrease-Key	1.15 ms	11.97 ms	3.69 ms	2.08 ms
	Lazy	2.34 ms	8.10 ms	6.74 ms	3.39 ms

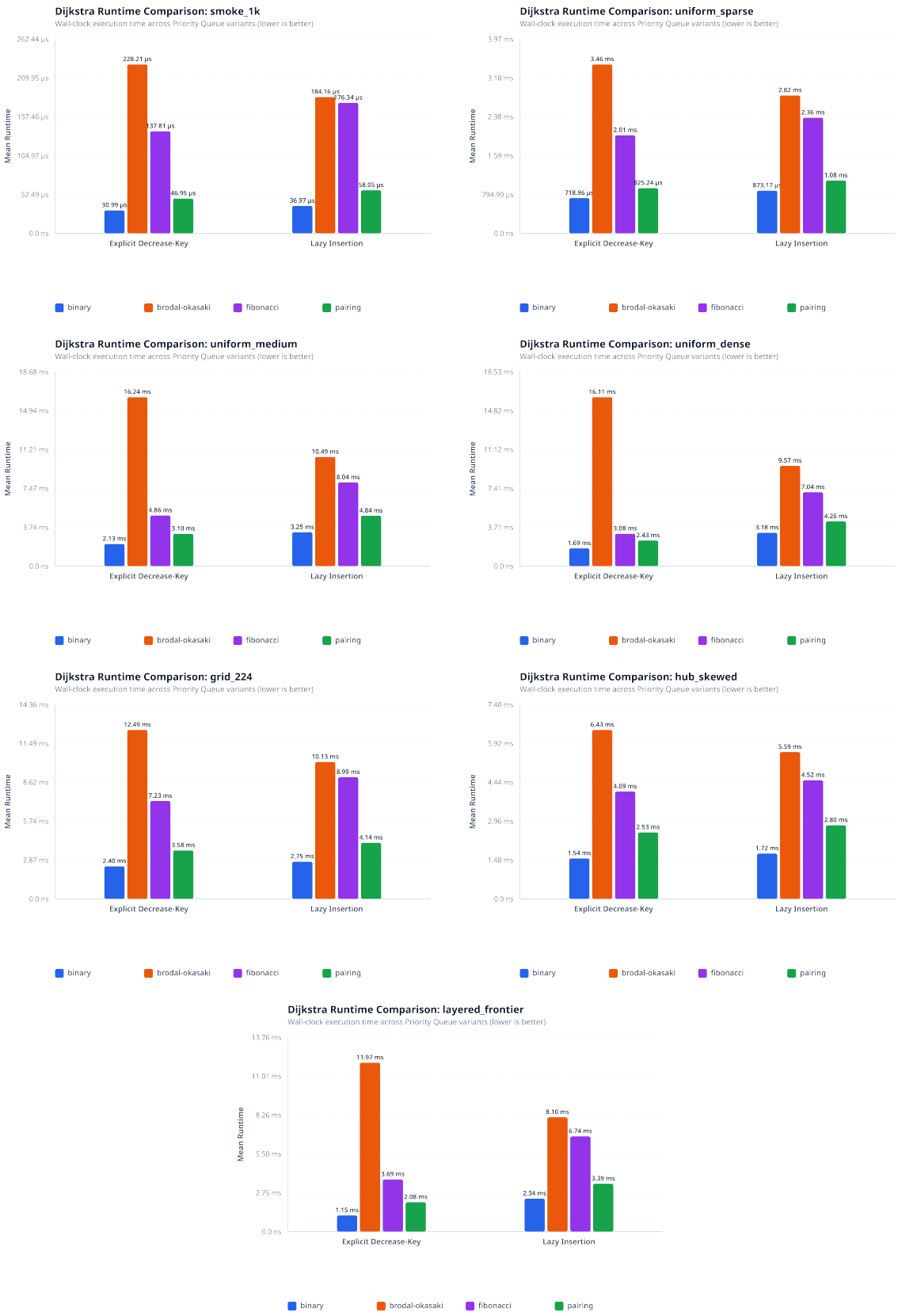


Figure 1: Runtime comparison between the four heap implementations across the seven input graphs. Each plot shows both `decrease_key` and `lazy` Dijkstra.

The binary heap is the fastest implementation in every graph configuration. The pairing heap is

consistently second, while the Fibonacci and Brodal-Okasaki heaps are slower. This ordering is stable across both the uniform random graphs and the structured grid, hub-skewed, and layered graphs.

For example, on `uniform_dense` with `decrease_key`, the binary heap takes approximately 1.69 ms, compared with 2.43 ms for the pairing heap, 3.08 ms for the Fibonacci heap, and 16.11 ms for the Brodal-Okasaki heap. On `grid_224`, the corresponding times are approximately 2.40 ms, 3.58 ms, 7.23 ms, and 12.49 ms.

The binary heap's advantage is consistent with its contiguous array layout, direct index-based navigation, and low constant factors. Although the Fibonacci and Brodal-Okasaki heaps offer stronger theoretical bounds for some priority-queue operations, their additional pointer manipulation and bookkeeping do not translate into lower end-to-end Dijkstra runtime in these experiments. The pairing heap is more competitive, but still remains slower than the binary heap on every input.

4.2.2 Runtime Comparison Between Lazy and Decrease-Key Dijkstra

The runtime difference between the two Dijkstra variants is shown in Figure 2. The figure reports the ratio $T_{\text{lazy}}/T_{\text{decrease_key}}$; values above one indicate that `decrease\_key` is faster, while values below one indicate that the lazy variant is faster.

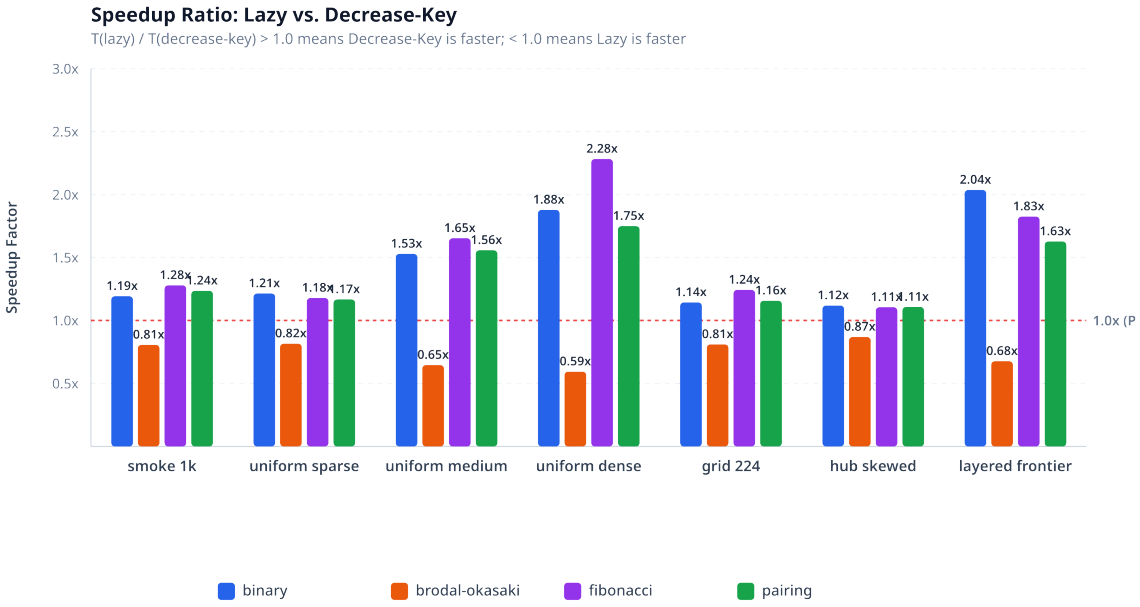


Figure 2: Runtime ratio between lazy and decrease-key Dijkstra. Values above one indicate that decrease-key is faster.

For the binary, pairing, and Fibonacci heaps, the `decrease\_key` variant is faster on every graph. The difference is small on the sparse and hub-skewed graphs, but becomes larger as the number of relaxations increases. For example, with the binary heap, lazy processing is approximately 1.21 times slower on `uniform_sparse`, 1.53 times slower on `uniform_medium`, and 1.88 times slower on `uniform_dense`. The same pattern appears for the pairing and Fibonacci heaps.

This behaviour is consistent with the lazy algorithm inserting duplicate entries whenever a shorter path is found. These stale entries remain in the priority queue until they are extracted and discarded, increasing the number of queue operations. The effect is particularly visible on `uniform_dense` and `layered_frontier`, where the larger number of relaxations produces a larger queue workload.

The Brodal-Okasaki implementation shows the opposite result. Its lazy variant is faster on all seven graphs, with the largest relative improvements on `uniform_dense`, `uniform_medium`, and `layered_frontier`. For example, on `uniform_dense`, lazy processing takes approximately 9.57 ms compared with 16.11 ms for `decrease_key`. This result is specific to the current implementation: because the heap has no handles, each decrease-key inserts a tombstone into the negative queue as well as the new entry, and both must later be removed by delete-min operations when they cancel, whereas the lazy variant inserts only the new entry (Section 5).

Overall, the results show that lazy processing is not uniformly inferior. For the binary, pairing, and Fibonacci heaps, explicitly maintaining decrease-key handles is beneficial, especially on denser graphs. For the Brodal-Okasaki implementation, avoiding decrease-key operations is consistently advantageous. Thus, the effect of the Dijkstra variant depends not only on graph density and structure, but also on how efficiently each heap implements decrease-key.

4.3 Hardware Performance Results

The hardware counters were collected with Linux `perf`, counting only the timed Dijkstra loop (Section 3.1). Because the graphs range from 1,000 to 50,176 vertices, the figures below divide each count by the number of vertices in the graph, which puts every graph on one axis while keeping the quantity proportional to its cost.

4.3.1 Work and Execution Efficiency

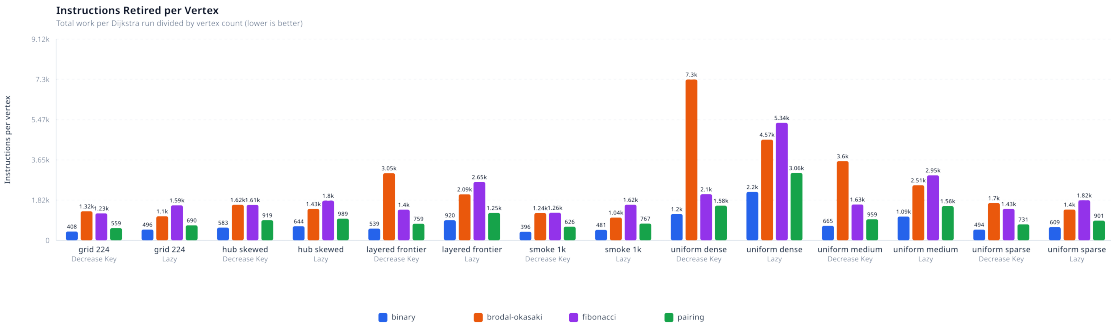


Figure 3: Instructions retired per vertex across heaps, input graphs, and Dijkstra variants.

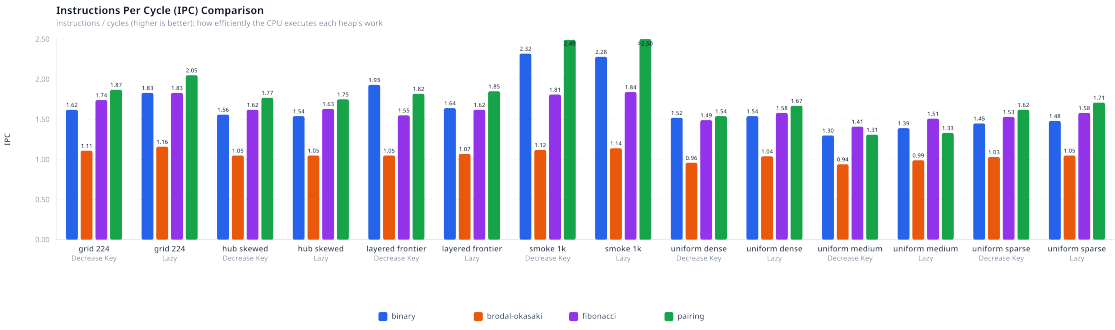


Figure 4: Instructions per cycle across heaps, input graphs, and Dijkstra variants.

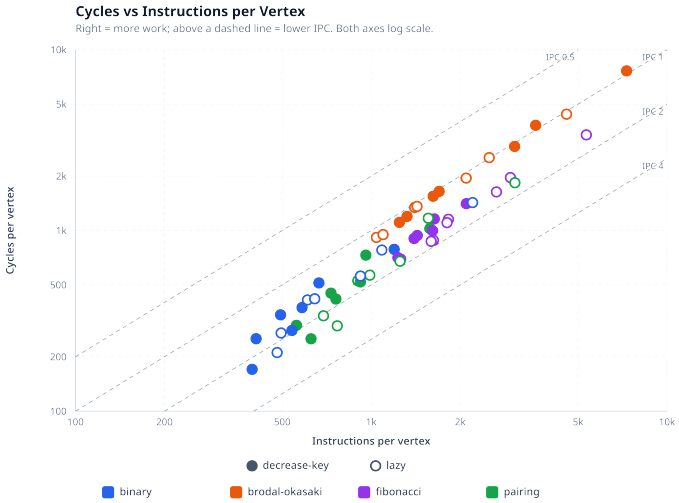


Figure 5: Cycles against instructions per vertex (log scales). Points further right do more work; points above a dashed line execute it at a lower IPC.

The slower heaps do substantially more work. Averaged across the seven graphs in `decrease_key` mode, the binary heap retires 612 instructions per vertex, compared with 876 for the pairing heap (1.4×), 1,522 for the Fibonacci heap (2.5×), and 2,833 for the Brodal-Okasaki heap (4.6×) (Figure 3). Lazy processing raises the count for the binary, pairing, and Fibonacci heaps (919, 1,317, and 2,541), matching their slower lazy runtimes, and lowers it for Brodal-Okasaki (2,020), matching its faster lazy runtime.

The binary, pairing, and Fibonacci heaps execute that work at a similar rate, with an average IPC of 1.67, 1.77, and 1.59 respectively (Figure 4). Among these three, the runtime ranking is therefore explained by the amount of work rather than by how efficiently the processor executes it, which appears in Figure 5 as these points lying along the same IPC band. Brodal-Okasaki is the exception: its average IPC is 1.04, so it both does more work and executes it less efficiently. On `uniform_dense` with `decrease_key` it retires 6.1× the binary heap's instructions per vertex but spends 9.7× its cycles (7,639 against 786).

These results do not support Hypothesis 2.2. The binary heap does execute the fewest instructions, but by a factor of 1.4 to 4.6 rather than orders of magnitude, and its IPC is not systematically lower than that of the pointer-based heaps: it is slightly below the pairing heap's and above the Fibonacci and Brodal-Okasaki heaps'.

4.3.2 Cache and Branch Behaviour

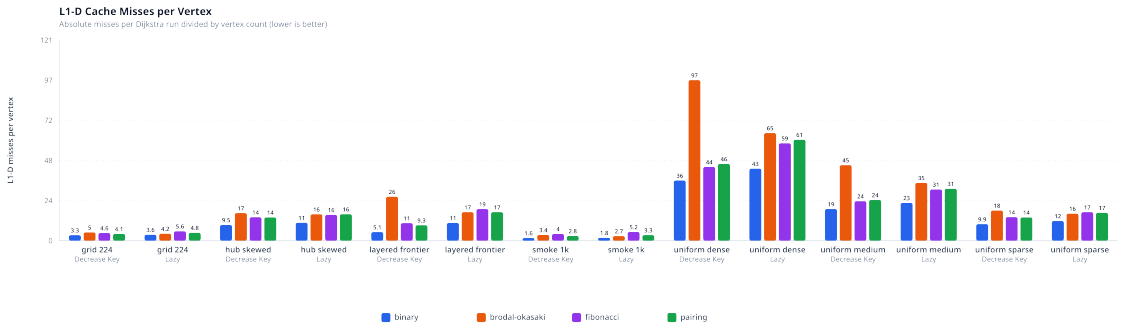


Figure 6: L1 data-cache misses per vertex across heaps, input graphs, and Dijkstra variants.

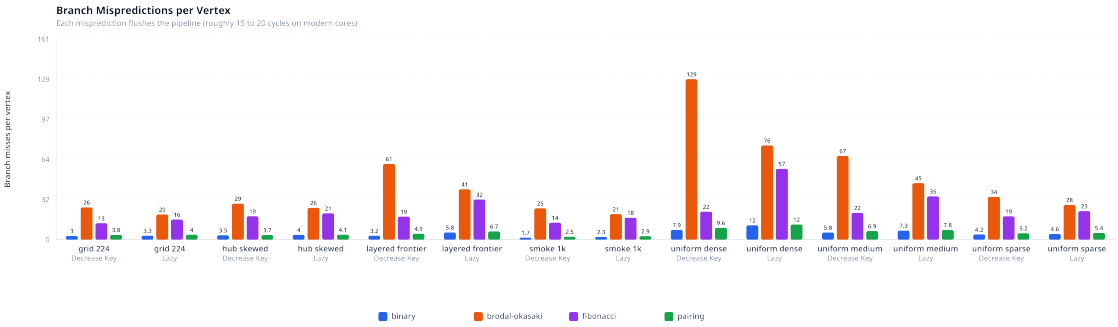


Figure 7: Branch mispredictions per vertex across heaps, input graphs, and Dijkstra variants.

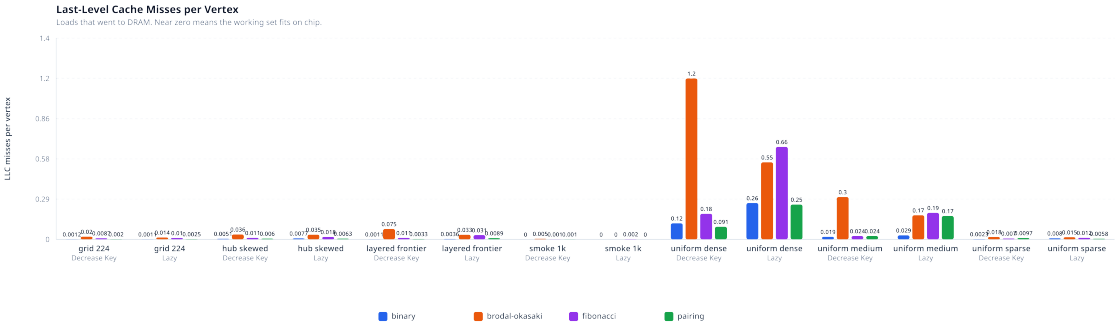


Figure 8: Last-level cache misses per vertex. Values near zero mean the working set stays on chip.

We report cache misses as counts rather than miss rates. A miss rate divides misses by loads, and the number of loads differs widely between heaps, so a heap that performs many extra loads that hit in the cache appears to have a better rate while costing more. Measured as a rate, the binary heap has the highest average L1 miss rate in `decrease_key` mode (7.04%, against 6.74% for pairing, 4.54% for Fibonacci, and 3.29% for Brodal-Okasaki), but it also performs the fewest loads (147 per vertex, against 211, 324, and 763). Counted per vertex, it has the fewest L1 misses: 12.1, against 16.4 for pairing, 16.5 for Fibonacci, and 30.3 for Brodal-Okasaki (Figure 6). This supports the first part of Hypothesis 2.1, that the array layout gives the binary heap the best cache behaviour, but not the predicted ordering of the pointer-based heaps: the Fibonacci heap's larger nodes do not make it worse than the pairing heap, and Brodal-Okasaki is the worst despite its contiguous buffer.

The cost of these misses is limited at our input sizes. Last-level cache misses peak at 1.2 per vertex (Brodal-Okasaki on `uniform_dense`) and stay below 0.05 in 43 of the 56 configurations (Figure 8), against 12 to 30 L1 misses per vertex on average, so almost every L1 miss is served from L2 or L3 rather than from main memory. The DRAM round-trips anticipated in Hypothesis 2.1 therefore do not occur in these experiments; the largest LLC counts appear on `uniform_dense`, whose working set is the closest to the 12 MiB L3.

Branch mispredictions separate Brodal-Okasaki most clearly from the other heaps (Figure 7). In `decrease_key` mode it averages 53.1 mispredictions per vertex (10.8% of branches), against 4.2 for the binary heap (4.0%), 5.2 for pairing (3.3%), and 18.2 for Fibonacci (5.2%). Its linking and cancellation code is driven by data-dependent key comparisons, which the branch predictor cannot learn. At the roughly 15 to 20 cycles each misprediction costs on modern cores, the 129 mispredictions per vertex on `uniform_dense` would account for a quarter to a third of its 7,639 cycles per vertex, which is consistent with its low IPC.

4.4 Memory Results

4.4.1 Peak Heap and Resident Memory

Figures 9 and 10 show the peak heap allocation and peak resident set size recorded during the experiments.

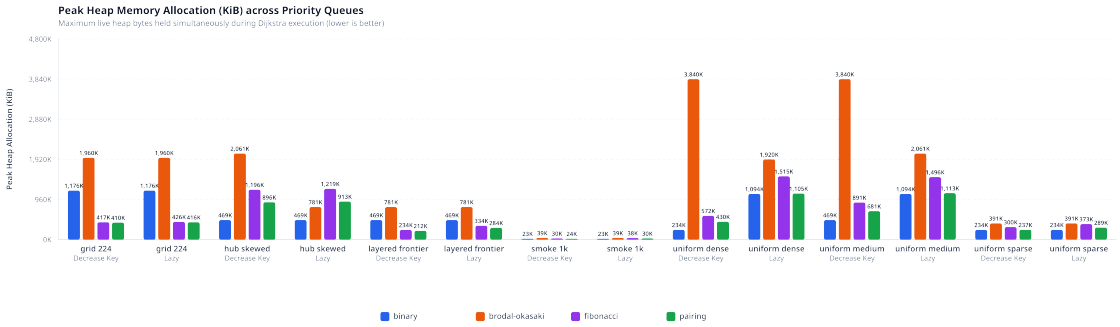


Figure 9: Peak heap allocation across heap implementations, input graphs, and Dijkstra variants.



Figure 10: Peak resident set size across heap implementations, input graphs, and Dijkstra variants.

In `decrease_key` mode the pointer-based heaps track the analytical node sizes of Section 2.1.2 closely. On `uniform_dense`, the pairing heap peaks at 430 KiB and the Fibonacci heap at 572 KiB, or 44 and 59 bytes per vertex against the predicted 48 and 64; they fall slightly short because not every vertex is in the queue at once. The binary heap allocates its 24 bytes per vertex up front and peaks at 234 KiB. It has the lowest peak in 10 of the 14 configurations. The exceptions are `grid_224` and `layered_frontier`, where the pairing and Fibonacci heaps use less because they allocate only the entries currently in the queue, and these two graphs keep a narrow frontier.

Brodal-Okasaki does not fit the prediction. Hypothesis 3.1 placed it second, behind only the binary heap, but it has the largest peak in 13 of the 14 configurations, reaching 3,840 KiB on `uniform_dense` in `decrease_key` mode, nine times the pairing heap. Its decrease-key never removes the old entry: it adds a tombstone to the negative queue and inserts the new entry, and neither cell is freed until the two cancel at the top of the heap. On `uniform_medium`, `uniform_dense`, and `hub_skewed` the live entries and tombstones outgrow the buffer's initial capacity of one cell per vertex, forcing it to grow, and its lazy variant needs less memory (1,920 KiB on `uniform_dense`, half the eager figure), the opposite of the predicted direction.

For the other heaps, lazy processing does increase peak memory, as predicted, but not in proportion to m. On `uniform_dense` the binary heap's peak rises from 234 to 1,094 KiB and the pairing heap's from 430 to 1,105 KiB, yet the total number of entries ever pushed (32,814) is only about one fifteenth of the 500,000 edges (Section 5).

Peak resident set size (Figure 10) lies between 49.9 and 52.6 MiB in every configuration. Each benchmark executable contains all seven input graphs as static data, which dominates the resident set, so RSS cannot distinguish the heaps and the fragmentation effect anticipated in Hypothesis 3.1 is not measurable at these sizes. No negative allocation values or leaks were recorded.

4.4.2 Allocator Activity

Figure 11 shows allocator activity across the heap implementations and input graphs.

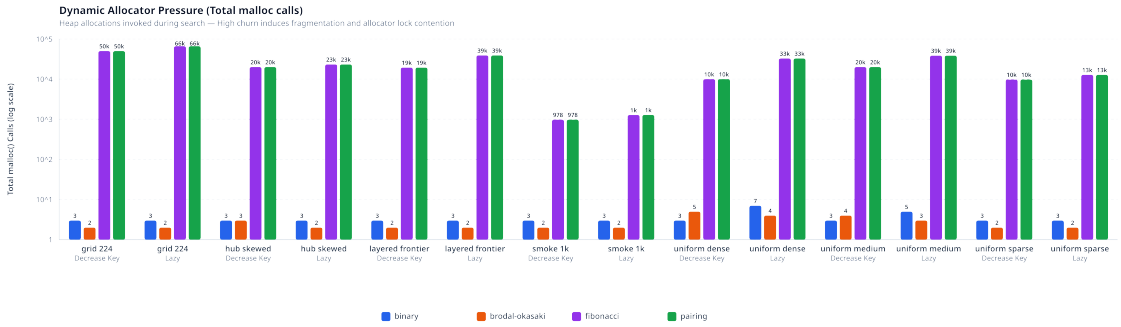


Figure 11: Dynamic allocation activity across heap implementations, input graphs, and Dijkstra variants.

The buffer-based heaps make at most seven allocator calls per run, as Hypothesis 3.2 predicted: the binary heap allocates its arrays once, and Brodal-Okasaki manages its own free list inside a single buffer. The pairing and Fibonacci heaps allocate one node per push. In `decrease_key` mode that is one node per reached vertex (10,000 on `uniform_dense`, plus two allocations for the heap structure and position array), also as predicted. In lazy mode it is one node per entry pushed, 32,814 on `uniform_dense`, against the prediction of roughly $m = 500{,}000$; Section 5 explains the difference. All configurations report matching allocation and free counts with zero leaked bytes.

These allocation counts are consistent with the runtime results: the pointer heaps pay for an allocator call on every push, and lazy processing adds more of them, while the two buffer-based heaps avoid the allocator almost entirely.

4.5 Hypotheses Revisited

Table 7 summarises each prediction from Section 2.2 against the measurements.

Table 7: Predictions from Section 2.2 compared with the results.

	Prediction	**Outcome**
1.1	Binary heap fastest on sparse graphs.	Supported: fastest on every graph in both variants.
1.2	Pairing overtakes binary on dense graphs; Fibonacci never beats pairing.	First part refuted (binary 1.69 against 2.43 ms on `uniform_dense`); second part supported.
1.3	Lazy competitive for binary on sparse graphs; severe slowdown for all heaps on dense graphs.	Partly: lazy is 1.2× slower for binary even on sparse graphs; on dense graphs the slowdown is 1.8 to 2.3×, and Brodal-Okasaki is faster lazily.
1.4	Brodal-Okasaki uncompetitive with imperative heaps.	Supported: slowest on every graph, 9.5× the binary heap on `uniform_dense`.
2.1	Binary has the fewest cache misses; Fibonacci the most.	Partly: binary has the fewest L1 misses per vertex, but Brodal-Okasaki, not Fibonacci, has the most; no DRAM traffic occurs.
2.2	Array heaps run at lower IPC but retire orders of magnitude fewer instructions.	Not supported: 1.4 to 4.6× fewer instructions, at a similar IPC.
3.1	Peak memory Binary < Brodal-Okasaki < Pairing < Fibonacci; lazy memory scales with m.	Partly: pairing < Fibonacci and binary usually lowest, but Brodal-Okasaki is the largest; lazy memory grows with pushes, not m, and shrinks for Brodal-Okasaki.
3.2	Buffer heaps make $O(1)$ allocations; pointer heaps about n (eager) and m (lazy).	Partly: the first two hold; lazy allocations are $1.2n$ to $3.3n$, far below m.

4.6 Cross-Experiment Analysis

Taken together, the runtime, hardware-counter, and memory results consistently favour the binary heap. Its contiguous array representation gives it the lowest runtime in every tested configuration, while also limiting allocation activity and peak heap usage. The pairing heap is generally the second-fastest, but its pointer-based structure introduces additional memory and allocation overhead.

The Fibonacci and Brodal-Okasaki heaps do not overcome their higher constant costs in this workload, despite their stronger theoretical bounds for selected priority-queue operations. The hardware

measurements support this result: the slower implementations retire more instructions and incur more cache misses per vertex, and Brodal-Okasaki additionally loses efficiency to branch mispredictions.

The choice of Dijkstra variant also matters. Decrease-key is faster for the binary, pairing, and Fibonacci heaps because it avoids stale queue entries. The Brodal-Okasaki implementation benefits from lazy processing because its decrease-key operation is comparatively expensive. Overall, graph structure and heap implementation interact substantially, so theoretical complexity alone does not predict the observed end-to-end performance.

5 What Surprised Us

The headline result is that the binary heap was the fastest implementation on every graph in both Dijkstra variants, and that the pairing heap never overtook it. Hypothesis 1.2 predicted a crossover on dense graphs, where the pairing heap's cheap decrease-key should dominate, yet on `uniform_dense`, with 50 edges per vertex, the binary heap still takes 1.69 ms against 2.43 ms. We expected the explanation to be cache locality. The counters show that locality helps, since the binary heap has the fewest L1 misses per vertex, but the larger reason was one we had not anticipated at all.

5.1 Decrease-key is rarely called

Hypothesis 1.2 assumed that on dense graphs the $m - n$ possible decrease-key calls would overwhelm the n extractions. Counting the calls tells a different story. The pointer heaps allocate exactly one node per push, so the allocation counts in Section 4.4 give the number of pushes in each variant, and the difference between the lazy and `decrease_key` counts is the number of decrease-key calls (Table 8).

Table 8: Decrease-key calls compared with the $m - n$ assumed in Hypothesis 1.2.

Graph	$m - n$	Decrease-key calls	Per vertex
`smoke_1k`	3,000	304	0.30
`uniform_sparse`	30,000	3,066	0.31
`uniform_medium`	180,000	18,764	0.94
`uniform_dense`	490,000	22,814	2.28
`grid_224`	149,632	16,029	0.32
`hub_skewed`	100,000	3,362	0.17
`layered_frontier`	138,400	19,811	0.99

On `uniform_dense` only 22,814 of the 490,000 possible calls happen, under 5%, and no graph reaches even 15%. Most relaxations fail because, with random weights, the first tentative distance found for a vertex is usually already close to its final distance. This matches a classical result: for random edge weights the expected number of decrease-key operations grows only like $n \log(m/n)$ [6], about 39,000 for `uniform_dense`, the same order of magnitude as we measured.

This one fact explains three of our wrong predictions. The pairing heap's cheaper decrease-key saves time on only about two operations per vertex, too few to pay for its pointer-based layout, so there is no crossover. The lazy variant never holds anywhere near m entries, since it makes at most 3.3 pushes per vertex, so lazy Dijkstra is 1.1 to 2.3 times slower rather than severely degraded (Hypothesis 1.3). And the pointer heaps make $1.2n$ to $3.3n$ allocations in lazy mode, not the roughly m predicted by Hypothesis 3.2. A graph that really favours a cheap decrease-key needs edge weights arranged so that relaxations keep succeeding, which uniform random weights do not produce.

5.2 Brodal-Okasaki's decrease-key is lazy Dijkstra with extra work

Brodal-Okasaki was the only heap that ran faster in lazy mode, on every graph and by up to 41% (9.57 against 16.11 ms on `uniform_dense`), and it used half as much memory doing so (1,920 against 3,840 KiB). The implementation explains why. A functional heap has no handles, so decrease-key cannot find and change an existing entry. Instead it inserts a tombstone for the old (distance, vertex) pair into a negative queue and inserts the new pair, and both stay in the buffer until they meet at the top and cancel. Every decrease-key therefore costs two insertions now and two delete-min operations later, where lazy Dijkstra pays one insertion and one delete-min for the same relaxation. In effect, the `decrease_key` variant is

lazy Dijkstra with tombstones added, so it can only be slower. This also overturns Hypothesis 3.1, which placed Brodal-Okasaki second-smallest in memory: it has the largest peak in 13 of the 14 configurations.

5.3 The heap with the fewest cache misses had the highest miss rate

Our L1 miss-rate figure ranked the binary heap worst, which seemed to contradict its speed. Counted per vertex, it has the fewest misses (12.1, against 30.3 for Brodal-Okasaki); its rate is high only because it makes so few loads (147 per vertex against 763). A rate divides by a denominator that differs between implementations, so it can rank them in the opposite order to their cost, which is why Section 4.3 reports counts instead.

5.4 Our measurements were wrong twice before they were right

The first hardware-counter results showed every heap running at an IPC close to 1 and retiring 26 to 28 million instructions on `smoke_1k`, even though the heaps' runtimes on that graph ranged from 30 to 230 microseconds. The counters were recording the whole process, including start-up and graph construction, rather than Dijkstra alone. With the counting window fixed, the binary heap retires 0.40 million instructions on that graph and Brodal-Okasaki 1.24 million. Later, a full rerun made the Fibonacci heap about three times slower on several graphs while the other heaps were unchanged. Dividing the `perf` cycle counts by the measured runtimes showed that the processor had run at about 1.7 GHz during those benchmarks, against roughly 4.8 GHz elsewhere: the laptop had throttled. Both errors produced plausible-looking results, and both were caught only by checking one measurement against another.

5.5 What held

Several predictions did hold. The Fibonacci heap never beat the pairing heap, as Hypothesis 1.2 expected of its cascading cuts and four-pointer nodes, and it suffers the most from lazy processing (2.28× slower on `uniform_dense`). Brodal-Okasaki was the slowest heap on every graph, supporting Hypothesis 1.4, although its slowdown is better described as a constant-factor cost of this implementation than as a consequence of the data structure itself. Once the counters were restricted to the Dijkstra loop, they explained the ranking clearly: the pairing and Fibonacci heaps execute 1.4 and 2.5 times the binary heap's instructions at a similar IPC, while Brodal-Okasaki executes 4.6 times as many at an IPC of about 1.0, held back by frequent branch mispredictions. Overall, the experiments show that for this workload, how often each operation is actually used matters as much as its asymptotic cost, and memory layout and constant factors decide the rest.

6 Future Improvement

It would be interesting to try extending this experiment across a wider range of heap implementations, especially focusing on making small modifications to each of the currently selected heaps. We could observe if different optimisations actually produce a real-world impact under the fairly complex runtime conditions of a min-heap. This would be particularly interesting for the Brodal-Okasaki heap, and to also add the imperative Brodal min-heap [1] as another point of comparison.

The present interface identifies a queue entry by its vertex number; this requires additional work to support `heap_decrease_key`, and the current index tables do not support multiple entries for a vertex alongside `heap_decrease_key`. Instead, each push could be assigned a unique identifier, used for looking up the node in `heap_decrease_key`, which would be a more correct implementation and remove the need for the positive and negative cancellation lists in Brodal-Okasaki that slow the implementation down. Since the edge case that would cause an incorrect `heap_decrease_key` does not arise under Dijkstra's algorithm, we decided to skip this feature.

Our hardware counters show that every working set stayed on chip: last-level cache misses never exceeded 1.2 per vertex. Running graphs whose working set is several times the 12 MiB last-level cache would test whether the binary heap's lead holds, or grows, once heap accesses have to go to main memory.

Finally, a direct test of Hypothesis 1.2 needs inputs that actually exercise decrease-key. Uniform random weights cause at most about two decrease-key calls per vertex (Section 5). Constructions in which most vertices are first reached through heavy edges and then improved repeatedly by lighter paths

can make almost every relaxation succeed, giving the pairing and Fibonacci heaps the workload their theoretical bounds are designed for.

7 Individual Contributions and Reflections

7.1 Oscar Chevalier

I worked primarily on the Brodal-Okasaki heap, both initially implementing it and optimising it. My next major role was to maintain the project more generally. I did all the work on the build system (Makefiles), ensuring the build process is flexible, correctly captures dependencies to trigger re-builds, and easy to maintain. In order to do this I spent time orchestrating the group towards end goals and providing starting points for others to work on, and then adjusting their work so that everyone could build and run the project, and that the layout was easy to understand. This lent me a strong understanding of the project as a whole, and the overall experimental design. Given this, I wrote the experimental methodology section of the report, and produced the video presentation slides.

The Brodal-Okasaki heap is a very complex design. While I understand the major design aspects of Brodal-Okasaki, I am not sure I fully understand the way in which those aspects interact and flow together. Specifically, I understand that the skew-binomial tree allows for $\mathcal{O}(1)$ `heap_push` operations, and much of the complexity comes from maintaining the invariants of skew-binomial tree links. However, looking at the functions and code that maintain those invariants makes this understanding go out the window, especially when it comes to removing the minimum element.

Further optimising it for cache locality made this even harder to understand. It was particularly difficult to get all the little helper methods correct during the rewrite, as it was hard to understand the role each one played in the larger picture, and easy to mix up the sibling and child addresses, causing a hard to find bug. In contrast, while cache locality was initially a concern, once I realised I could allocate all the cells in a continuous buffer and use buffer-relative addressing, the problem seems to have been addressed rather well, and it was a fun solution to the problem.

The worst cases for Brodal-Okasaki are made very apparent in this experiment, `decrease_key` is a nightmare! Supporting it was a lot of extra work, and not a part of the original paper due to how unfriendly the operation is to persistent data structures. I believe there are other approaches to support `decrease_key` in persistent min heaps, but the negative queue idea seemed appealing to try. The additional invariants that needed to be maintained around negative queue elements added significant complexity to the design.

If starting the project again, I think I'd pick the regular Brodal queue, which is also a skew-binomial tree, but more similar to the fibonacci heap, using parent links and double links to improve performance. It would also be interesting to apply similar memory management approaches to the fibonacci and pairing heaps, and see if better memory management techniques could allow one of the high allocation heaps to beat the binary heap.

Maintaining the project build was a secondary but also interesting task. I already knew how to use make, but this was a lesson in maintaining a larger and more complex build with it. Make is probably the only build system that can do semi-incremental builds and can do such a complex arrangement of inputs and outputs, using and invoking a large range of system tools, however at this level of complexity it was clear that make was barely holding up. I needed to rewrite large parts of the Makefile 4 times as the structure of the project changed, and we added more components. Make would do well to be replaced with a more capable programming language while still supporting the ability to interact well with a wide range of other programs. When issues occurred due to variable expansion or similar it was hard to debug the Makefile. Extending this project would make me seriously consider writing something to replace make. We also temporarily considered `nob.h` or `nobuild.h` which only uses c to build c code, however it did not seem mature enough. Still, it beats gradle any day of the week.

Overall this was a lot of fun, and the broad scope of the project brought a lot of interesting challenges to solve with a set of tools I do not usually use, which is always a good way to expand your skillset.

7.2 Truong Giang Nguyen

I implemented the base Dijkstra's algorithm for benchmarking. Since C has no dynamically sized array, I could not simply represent the graph using a normal adjacency matrix. Implementing one from scratch could introduce allocation overhead unrelated to the benchmark, so I implemented the shared graph representation using Compressed Sparse Row (CSR) format.

My algorithm is the binary heap. It has logarithmic time complexity for push, pop, and decrease-key, and offers constant-time minimum queries. The worst case happens when relaxation keeps succeeding and constantly pushes new lower values onto the heap. For the lazy variant, this causes the heap to store a large number of stale entries and grow in size, making the bubble-up and sink-down operations longer. If every newly encountered edge adds a new entry to the heap, the result is $\mathcal{O}(\log m)$ push and pop operations, where m is the number of edges. Stale entries also cause more pops, which wastes computation just to reach a valid entry. The second variant, eager, mitigates this by calling `heap_decrease_key()`, which directly alters the entry instead of pushing a new one. This approach, however, requires an extra index array to store each entry's position in the heap. The number of successful relaxations is positively correlated with graph density, as more edges come in and out of each vertex. Although the worst case can be crafted manually by deliberately placing edge weights in a certain order, it is extremely rare with randomised input. As a result, the binary heap won all test cases because of its contiguous data allocation, which gives it a large advantage in cache locality.

The hardest bug I encountered was in `heap_pop`. The assertion I wrote fired in `heap_decrease_key` instead, so I spent a lot of time around the wrong function. I decided to use Claude Code to review my code, and it turned out I copied the last element to the root before decrementing `size`, so `h->data[0] = h->data[h->size]` read the slot one past the end. This only corrupts the heap when that stale slot happens to hold a value smaller than the true minimum, so small graphs passed while large ones returned wrong distances. The fix was simply to decrement first, then copy.

A key design decision I made was the `pos[]` array mapping for the vertices. Without it, decrease-key would need a $\Theta(n)$ scan to find the corresponding key. I had to refactor almost every function in the binary heap implementation, as all operations need to update and keep track of the position of each added or removed node. It also introduces an extra memory allocation for the array, which affects the lazy variant even though it never reads it.

If I were to do it differently, I would split the heap into two versions: `lazy_binary_heap`, which does not support `decrease_key`, and `eager_binary_heap`. This would give me the rawest benchmark of the lazy variant of Dijkstra's algorithm. I would then compare it to C++'s `std::priority_queue`, as it is also implemented as a binary heap and does not support `decrease_key`.

7.3 Tung Nguyen

Beyond implementing both the pairing heap and Fibonacci heap, my primary group contribution was designing and running the benchmarking suite. I planned the experimental methodology across the three experiments:

- Wall-clock execution time with `ubench`
- Hardware performance counters using Linux `perf stat`
- Dynamic memory footprint via linker-wrapped allocator tracking.

I developed the benchmark execution pipelines, collated the experimental output into structured CSV datasets, and integrated the generated figures and result tables into the report.

7.3.1 Pairing Heap

The worst case for the pairing heap occurs during Delete-Min. In this implementation, removing the root first pairs its children from left to right and then melds the resulting trees from right to left. Therefore, an input that creates a large root with many children, such as a dense graph with many frontier relaxations, can make one extraction expensive. Repeated decrease-key operations can also create irregular tree shapes because a decreased node is cut from its current sibling list and melded back with the root.

The hardest design issue was maintaining the left-child/right-sibling representation correctly. Each node stores a `child`, `next`, and `prev` pointer. But, `prev` has two meanings as it refers to the parent when the node is the first child, and to the previous sibling otherwise. This made cutting a node during Decrease-Key particularly error-prone. A mistake in either case could disconnect part of the tree or leave an invalid pointer for the next operation. I resolved this by treating the two cases separately and updating both the predecessor and successor whenever a node was removed.

If I implemented the pairing heap again, I would define explicit helper functions for unlinking a node and for appending a child before writing the main operations. I would also add invariant checks after

every operation, including checks that the root has no parent, sibling links are symmetric, and every node appears in the position array exactly once. This would make pointer errors easier to isolate than debugging them through a complete Dijkstra run.

Before this project, I did not fully understand the purpose of MELD in a pairing heap. I now understand that it is the core operation used to combine trees during insertion, DECREASE-KEY, and DELETE-MIN. By linking the larger root under the smaller one, the heap preserves the heap-order property without requiring global rebalancing.

7.3.2 Fibonacci Heap

The worst case for the Fibonacci heap is associated with EXTRACT-MIN and cascading cuts. Extracting the minimum can require consolidating many roots of equal degree, while a sequence of decrease-key operations can repeatedly cut nodes from their parents and propagate cascading cuts up the tree. Dense graphs with many successful edge relaxations are therefore a natural stress case because they produce frequent decrease-key operations followed by consolidation.

The most difficult part was maintaining the circular doubly-linked root and child lists. Every insertion, cut, child promotion, and tree link modifies several links at once. In particular, EXTRACT-MIN must promote all children of the removed node, remove that node from the root list, and then rebuild the list after consolidation. I also had to preserve the marked-bit invariant, where the first child loss marks a node, while a later loss cuts it and continues the cascading cut. I resolved these issues by isolating linking and cutting into helper functions and by carefully resetting parent and mark fields when nodes became roots.

If I started again, I would first implement and test the circular-list operations independently before integrating them with Dijkstra. I would also avoid relying on a fixed `MAX_DEGREE` without documenting the bound and would use a dynamically sized consolidation array based on the heap size.

I learned that Fibonacci heaps achieve their theoretical advantage by postponing work rather than eliminating it. At first, I was tempted to optimise the implementation to make every operation appear efficient. But I eventually realised that the main constraint is the trade-off between cheap insertions and decrease-key operations and the more expensive restructuring performed during EXTRACT-MIN. I accepted this trade-off as an essential part of the Fibonacci heap rather than trying to eliminate it.

8 AI Disclosure

Each individual experience contains longer reflection on cases where the AI tools used in this project went wrong, summarised here:

Oscar LLMs generated brittle code, hardcoding the names of graph inputs, or generating tests that were tuned to only work on a specific CPU, causing portability issues.

Oscar The LLM generated perf tests were incorrectly measuring program startup events, and the logic to measure only the correct sections of the code was half implemented.

Oscar LLM recommended figures were misrepresenting the data, or not presenting measured data.

Oscar LLM adjustments to the Makefile without specific instructions made it unmaintainable.

Giang AI overengineered solutions when given a non-specific request, like adding the `heap_decrease_key` Dijkstra's variant.

Giang AI generated presentation slides were noisy, hallucinated benchmark figures, and tried to pack too much detail into a 15 minute presentation.

Giang AI overlooked pre-existing features, generating more code to solve data parsing problems that did not exist.

Tung AI was overenthusiastic about suggesting hardware tests that the team was unfamiliar with, and when implementing them would cut corners, and skip test inputs already present in the project, being exceptionally lazy.

Tung LLM analysis of the produced data produced inaccurate claims and introduced pre-conceived biases into the writing, rather than accurately describe the findings.

Tung A second attempt at generating presentation slides with a different agent caused the same over complication issues for a 15 minute presentation.

8.1 Oscar's Experience

This was my first time using AI in any meaningful manner. There was both good and bad.

When I knew exactly the changes that I wanted LLMs were very helpful to quickly make changes that would have otherwise been tedious. Certain tasks were not particularly appealing to work on, but using LLM generated code and auditing it meant that I could quickly get to a place where I was happy. Notable parts of the project that were AI generated are the perf and mem tests, some of the graph generation logic, the plot generation, and the presentation.

On the other hand, when not enough specific guidance was given, or if another group member who had a less clear image of the outputs they wanted had prompted the LLM, I'd often find that the produced code was overly brittle. For instance, the early implementations of perf recorded intel specific events, and were not portable at all. The early perf and mem tests also hardcoded the graph and heap names, and would not have held up well to changes in the inputs.

Later, we discovered that the perf tests were only half configured to measure just running Dijkstra's algorithm with the heaps, and were actually also capturing all the events during program startup, including loading all the graphs into memory. This bug was only discovered due to review with an LLM and rendered the results completely meaningless. The same review session revealed that the data that we were presenting from the perf tests was a misrepresentation of what was actually happening, and that the report did not cover the branching events, or acknowledge that the data was not all it immediately seemed. This oversight was no doubt introduced because we are not individually experienced and competent at working with tools like perf, so we could not see for ourselves that there were issues.

However, despite these downfalls, it was easy to later correct the LLM, modifying the scripts to not hard code the systems by pointing them to good examples and motivations for why tasks should be completed in the way I wanted them to be done.

One aspect I did not like LLMs touching was the Makefile. They were all too happy to forever tack on additional targets and variables without considering how to integrate with the existing structures. Often a group member would ask the LLM to quick and dirty add their outputs to the Makefile, and then I would review and restructure the project's build process as the requirements and structure changed. The LLM Makefile contributions were always terribly implemented and completely replaced when I would manually audit the file.

Outside of generation, I found that using an LLM was very helpful for analysing how to optimise the Brodal-Okasaki heap. Claude identified that meld is not used, so bootstrapping is no longer necessary, and gave me the idea to put all cells in a buffer by suggesting that I run my own free list for the allocations, which is not quite the same thing. After I had rewritten the min heap, Claude was able to quickly catch a number of small mistakes I had made in the rewrite, before I wasted time running and debugging it. I quite enjoyed this explorative and feedback usage, where the LLM did not generate code for me, but was instead used to get a different perspective, or to simplify the reasoning about a very complex implementation.

For my first experience interacting with LLMs on a larger scale in a project, I was quite impressed with how much time it saved the group, but felt I always needed to spend time properly posing questions to the LLM and reviewing the outputs, as there would always be something wrong with the first attempt. I suspect recent improvements in the quality of tools means I'll need to begin taking them more seriously, and come to terms that they will be part of software development going forward.

8.2 Truong Giang's Experience

I used Claude Code to scaffold the project structure, since my task is the very first step in the project. Initially, Claude implemented a vector to use as adjacency list for the graph representation. Since our project focuses on benchmarking the heap implementation, I wanted to minimise the impact of other operations outside of the heap. So I ask Claude to change that to a more effective format, so it suggested using CSR table. I then announce my group about this change before approving it.

One thing I found AI to be particularly helpful is when debugging pointers usage and memory allocation. Since C offers raw control over the allocated memory space, it is very easy to miscalculate the required size for the allocation, eventually leading to a silent crash. I encountered a bug in the binary

heap's `heap_pop` function, but the assert failed in the `heap_decrease_key` function instead. Claude Code instantly caught this, saving me a lot of time trying to debug it manually.

I noticed that AI tends to overly engineer the solution when the given input is not specific enough. An example is when we decided to implement another variant of the Dijkstra's algorithm. Since we have a single interface to call the Dijkstra's algorithm, it initially duplicated to another file, then rely on Makefile to dynamically link different objects to the selected algorithm. We then simplified it to have just a single file containing both variants.

It also hallucinates and overcomplicate the approach on more complex tasks. We used Claude to generate the presentation slides based on our report, the codebase, and the video requirement. This task requires the AI to review everything we have in our repository. Initially, it included some random benchmark figures that does not even exist in our codebase, and it tries to fit too much detail in a 15 minutes video. I then learnt that using AI effectively is not just about giving it as much information as possible, but also to limit on the scope of the problem.

When generating the benchmark visualisations, AI overlooked the fact that the benchmarking library already supported exporting results directly to `.csv` files. Instead, it created a separate Python parser to read the output and convert it into a pandas DataFrame. This added unnecessary complexity to the workflow. We eventually discarded that implementation and provided AI with more context about the library's existing CSV export functionality so it could use it instead.

8.3 Tung's Experience

First, I started by digesting this assignment description with Gemini Flash 3.8. I asked it to point out the main pain points to look out while doing this project. Once that is done, I relay my understanding to the team. After that, I chose to implement Pairing and Fibonacci heaps. I then ask Gemini to briefly introduce me to these heaps and provide good source on YouTube. I then check my understand with Gemini again by draft up the pseudocode and asking it to correct any mistake. Since these heaps were quite popular, the discussion session with the AI was fruitful as I can cross check myself with the AI and other sources, in which it also provided. So far, the AI has done a good job of helping me to catch up with the knowledge of this project.

Second, I moved on to implement the code for these heaps, and moved on to using Antigravity with Gemini 3.8 Flash. I wrote the code on my own and then ask the AI to fix it. The AI was very efficient with writing code, it did struggle with syncing my code to the rest of the codebase. Basically, instead of defining two parameters, we group it into an `Item`. But, my AI just code out the two parameters individually, a minor mistake and was easily self-checked and fixed. Later on, the we need to add a `heap_decrease_key` codebase wide. This was a repetitive task, in which the AI was very good at.

Third, I then plan ahead for the benchmark, i.e. what kind of experiment we should do. The AI was good here since it suggest hardware test, in which I'm not that familiar with. However, it went overboard and just listed a lot of things. After a team meeting, we only pick some of what it suggested, with the constraint of keeping thing not being too convoluted. I then had a back and forth session with Gemini on Antigravity on design the pipeline for each experiment, which went smoothly. I then ask the AI to implement these pipeline in the codebase. Here, the AI also kept looking for shortcut and was cutting down on test input. I had to explicitly tell it to *not* be lazy, still it did not follow through. It was not until the other day, when me and Oscar were getting together to do some code review, where Oscar was checking the Makefile and looking through the output manually. Here, he pointed out the missing test output from the AI generated pipeline. We were able to track down to the missing input, in which we prompted the AI to simply add them back. In the process, the AI was messing up the Makefile, causing Oscar to have to spend time fixing it. Later on, I switch to GitHub Copilot with ChatGPT 5.6 Sol as backend for better git integration. We used GitHub Copilot to fix what Gemini has done. Oscar and I were going back and forth to locate the error in the Makefile naming scheme, and put together a solid prompt for Copilot, in which at the end it fixed the error. Once that is out of the way, I ask Copilot to reorganise the output directories, convert them to `png` for LaTeX import.

Fourth, I moved on to writing the report. With the help of Copilot, I was able to import the charts smoothly. I also asked the AI to read my results analysis and cross-check with the output. However, the AI cross-checked against the graph images than the raw numerical tables, leading to inaccurate claims. For example, it assumed eager decrease-key was universally faster across all heaps, overlooking that the Brodal-Okasaki implementation was actually substantially faster under lazy Dijkstra across all seven graphs. It also failed to notice that Oscar's contiguous buffer optimisation eliminated allocator pressure for Brodal-Okasaki despite its higher runtime. We only noticed these errors when manually verifying the

text against the frozen CSV data, after which we re-prompted the AI with strict numerical references.

Lastly, I asked the AI to make HTML files for presentation. They contained too much information. So we scrapped it and Oscar redesigned it with Claude.

8.4 AI in Group Work

While some of our LLM usage was individual, a lot of it was group work. The team would work to coordinate our LLM usage and let each other know when we had changed a shared part of the project.

In particular, the Makefile was clearly a hot resource. As the project grew, the Makefile evolved too, often the LLMs would eagerly introduce poor practices into the Makefile, and inter-group communication allowed Oscar to examine and improve the LLM generated mess.

The report was a similar contested part of the project, which had a fair amount of LLM contribution. Team members would use LLMs to review the report, and then report the key findings to the rest of the group, allowing the team to plan how issues would be handled, and by whom. At the same time, LLMs produced invalid analysis several times, and it required cross checking by group members to ensure we were all happy with its validity.

LLMs would also encourage small ideas to quickly snowball out of control, like testing for a larger number of hardware events, or adding very time-expensive massive graph inputs to the tests. Having a group to review ideas with allowed us to review and cull ideas and tests that were not actually useful, but a solo developer might not have realised.

Keeping in contact with each other and strong individual task delegation allowed us to use LLMs to accelerate the development process and extend the project beyond what we could have done without them, however this definitely meant that we did not review and understand each others' code as well as we would have otherwise.

References

[1] Brodal, G. S. Worst-case efficient priority queues. In *Proceedings of the Seventh Annual ACM-SIAM Symposium on Discrete Algorithms* (Philadelphia, PA, USA, 1996), SODA '96, Society for Industrial and Applied Mathematics, pp. 52–58.

[2] Brodal, G. S., and Okasaki, C. Optimal purely functional priority queues. *Journal of Functional Programming 6*, 6 (1996), 839–857.

[3] Dijkstra, E. W. A note on two problems in connexion with graphs. *Numerische Mathematik 1* (1959), 269–271.

[4] Fredman, M. L., Sedgewick, R., Sleator, D. D., and Tarjan, R. E. The pairing heap: A new form of self-adjusting heap. *Algorithmica 1* (1986), 111–129.

[5] Fredman, M. L., and Tarjan, R. E. Fibonacci heaps and their uses in improved network optimization algorithms. *Journal of the ACM 34*, 3 (1987), 596–615.

[6] Noshita, K. A theorem on the expected complexity of Dijkstra's shortest path algorithm. *Journal of Algorithms 6*, 3 (1985), 400–408.

[7] Williams, J. W. J. Algorithm 232: Heapsort. *Communications of the ACM 7*, 6 (June 1964), 347–348.